

C++ Programming Textbook Notes

1. Overview of Constants in C++

Conceptual Explanation

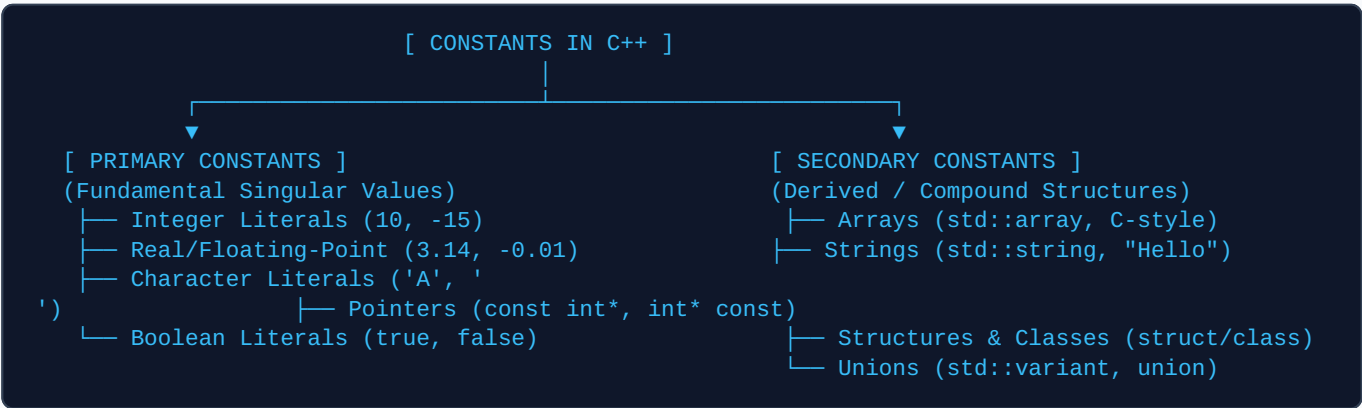
A **Constant** is an immutable value that remains completely fixed throughout the execution lifecycle of a program. Once defined, its value is "locked" in memory or evaluated directly during compilation.

While a **Variable** acts like a re-writable card box where stored values can be replaced at any point, a **Constant** acts like a sealed glass display case—once created, its value cannot be modified.

Variable vs. Constant Comparison

Feature	Variable	Constant
Mutability	Can change during program execution	Cannot change during program execution
Memory State	Contents vary (e.g., <code>age = 20</code> - <code>> 21</code>)	Contents remain strictly locked
C++ Code Syntax	<code>int age = 20;</code>	<code>const float PI = 3.14f;</code> or <code>constexpr double PI = 3.14159;</code>
Re-assignment	<code>age = 21;</code> (Allowed)	<code>PI = 5.0f;</code> (Compile Error)

Classification Hierarchy of Constants



2. Primary Constants in C++

Conceptual Explanation

Primary constants represent singular, literal values embedded directly into code instructions or assigned to read-only memory variables. Modern C++ classifies primary constants into four core literal categories:

- **Integer Constants:** Whole numbers without fractional parts (e.g., `10`, `-15`, `0`).
- **Real (Floating-Point) Constants:** Decimal numbers containing fractional parts (e.g., `3.14f`, `-12.35`).

- **Character Constants:** Single characters enclosed in single quotes (e.g., 'A', '7', ' ').
- **Boolean Constants:** Modern C++ built-in truth literals (true and false).

Modern C++ Constant Mechanisms: const vs constexpr vs consteval

Unlike standard C which primarily relied on const or macro definitions, modern C++ introduces compile-time evaluation keywords:

- **const:** Evaluated at **runtime or compile-time**; guarantees read-only access after initialization.
- **constexpr (C++11):** Guarantees **compile-time evaluation**, allowing constants to be embedded directly into program binary instructions without runtime memory fetch overhead.
- **constexpr (C++20):** Immediate functions that **must** produce a compile-time constant expression.

Text-Based Diagram: Runtime vs Compile-Time Constant Allocation

```
Runtime Variable (int x = 10):
RAM Address [0x7ffd9000] → Reads value from physical memory location at execution time.

Compile-Time Constant (constexpr int Y = 10):
Compiler Assembly Code → Replaces symbol 'Y' directly with literal value 10 during compilation.
                        (Zero RAM fetch overhead at runtime!)
```

Code Example

```
#include <iostream>

int main() {
    // 1. Integer Literal Constant
    constexpr int maxAttempts = 3;

    // 2. Real / Floating-Point Literal Constant
    constexpr double pi = 3.1415926535;

    // 3. Character Constant
    constexpr char passGrade = 'P';

    // 4. Boolean Constant
    constexpr bool systemReady = true;

    std::cout << "Max Attempts: " << maxAttempts << "
";
    std::cout << "PI: " << pi << "
";
    std::cout << "Grade: " << passGrade << "
";
    std::cout << "System Ready: " << std::boolalpha << systemReady << "
";

    // maxAttempts = 5; // COMPILE ERROR: Cannot reassign read-only / constexpr variable

    return 0;
}
```

⚠️ **Common Gotcha:** Forgetting to initialize a `const` or `constexpr` variable upon declaration (e.g., `const int age;`) will trigger an immediate compilation error.

💡 **Best Practice:** Prefer `constexpr` over `const` for values known at compile-time to allow compiler optimizations and eliminate runtime overhead.

3. Secondary Constants in C++

Conceptual Explanation

Secondary constants are structured, derived data types formed by grouping or referencing primary constants. Modern C++ provides safer and object-oriented abstractions over raw C-style secondary structures.

A. Arrays (`std::array` & Modern C++ Arrays)

An array is a contiguous block of memory holding homogenous primary constants or values under one identifier name. Modern C++ introduces `std::array` alongside standard C arrays.

```
Memory Layout of Array 'marks':
Index:    [0]    [1]    [2]    [3]    [4]
Value:    | 90 | 85 | 80 | 95 | 88 |
RAM Addr: 0x10  0x14  0x18  0x1C  0x20  (Contiguous 4-byte int steps)
```

B. String Constants (`std::string` & String Literals)

String constants represent sequences of characters. In C, string literals end with a hidden null-terminator byte (`'\0'`). Modern C++ introduces `std::string` and `std::string_view` for memory-safe string operations.

```
C-Style String Literal in Memory ("Gaurav"):
| 'G' | 'a' | 'u' | 'r' | 'a' | 'v' | '' | → Hidden null terminator signals string end
```

C. Pointers & Constant Pointer Mechanics

A pointer stores the memory address of another variable. In C++, pointer constness splits into two crucial rules:

- `const int* ptr`: **Pointer to Constant Data** (Cannot modify the target value through `ptr`).
- `int* const ptr`: **Constant Pointer** (Cannot point to a different address).
- `const int* const ptr`: **Constant Pointer to Constant Data** (Neither address nor target value can change).

Pointer Memory Visualization:
Variable X:

Value: 10
Address: 0x1000

Pointer PTR:

Value: 0x1000 (Addr of X)
Address: 0x5042



D. Structures (`struct`) & Classes (`class`)

Structures allow custom aggregation of heterogeneous data types into a unified contiguous memory block. In C++, structs are expanded to support constructors and member functions, behaving identically to classes except default member access is `public`.

Student Struct Memory Layout:

age (int - 4 Bytes)	marks (float - 4 Bytes)	grade (char - 1 Byte)
---------------------	-------------------------	-----------------------

E. Unions & Modern Type-Safe Alternatives (`std::variant`)

A union allocates a single shared memory location overlapping all member variables. Only one member value can be safely read at a time. Modern C++ provides `std::variant` as a type-safe alternative to raw C unions.

Union Shared Memory Space:

Shared Byte Space (Max Member Size) → Stores int 'i' OR float 'f'	(Overwriting 'i' corrupts 'f')
--	--------------------------------

Comprehensive Secondary Constants Code Example

```
#include <iostream>
#include <array>
#include <string>

// Secondary Constant: Struct Definition
struct Student {
    int age;
    float marks;
    char grade;
};

// Secondary Constant: Union Definition
union DataContainer {
    int intVal;
    float floatVal;
};

int main() {
    // A. Array (Modern C++ std::array)
    constexpr std::array<int, 5> marks{90, 85, 80, 95, 88};

    // B. String Constant
    const std::string name = "Gaurav";

    // C. Pointer Mechanics
    int value = 10;
    const int* ptrToConst = &value; // Cannot modify value through pointer

    // D. Struct Initialization
    const Student s1{20, 89.5f, 'A'};

    // E. Union Usage
    DataContainer container;
    container.intVal = 42;
```

```
std::cout << "Array Item [0]: " << marks[0] << "\n";
std::cout << "Student Name: " << name << ", Grade: " << s1.grade << "\n";
std::cout << "Pointer targets value: " << *ptrToConst << "\n";

return 0;
}
```

 **Pointer Constness Confusion:** Reading `const int* p` vs `int* const p` trips up beginners. Remember the *right-to-left rule*: read from right to left starting from the pointer name.

 **Union Overwrites:** Accessing a union member that wasn't the most recently written member results in undefined behavior or corrupted bits.

 **Best Practice:** In C++, prefer `std::array` over raw C-style arrays, `std::string`/`std::string_view` over raw character arrays, and `std::variant` over raw C unions for type-safe memory execution.